



## 计算力学作业报告

指导教师：郭然、肖姚

姓 名：查继鹏 202311012106

班 级：工程力学 231 班

学 院：建筑工程学院

时 间：2026 年 6 月 16 日

## 1、作业背景

本次程序设计作业对应课程《2.4 平衡方程组的求解》教学内容，核心研究有限元平衡线性方程组

$$Ka = R$$

课程核心知识点包含对称正定矩阵直接解法、 $LDL^T$  分解算法、活动列/轮廓稀疏存储、缓存友好型代码实现、多载荷工况批量求解，同时探究矩阵条件数、残差、离散舍入误差对有限元计算精度的影响规律。

本作业是前序《2.3 系统总体刚度方程》程序作业的延伸与拓展：

2.3 作业完成有限元前处理：从节点、单元、材料、边界约束、外载荷输入，生成单元刚度矩阵、组装整体刚度矩阵，划分自由/约束自由度，建立缩减平衡方程组。

本次 2.4 作业核心任务：为 2.3 输出的平衡方程组开发专用线性求解框架，自主实现  $LDL^T$  稠密求解器，并接入工业级稀疏求解器 Intel MKL PARDISO，完成小规模桁架、大规模 Poisson 有限元两类算例求解，定量分析求解精度、残差、条件数、运行效率与离散收敛误差。

## 2、 $LDL^T$ 分解算法说明

### 2.1 数学理论

对称正定方阵  $K_{n \times n}$  唯一分解形式：

$$K = LDL^T$$

L：单位下三角矩阵，对角线全部为 1；

D：正定对角矩阵，所有对角元  $D_{jj} > 0$ ；

对比 Cholesky 分解，本算法无平方根运算，降低浮点数值误差，适配有限元平衡方程。

### 2.2 迭代计算公式

来自代码文件 `ldlt_solver.py` 中的 `ldlt_factor` 函数

按列循环  $j=0, 1, 2, \dots, n-1$ ：

#### 2.2.1 对角元计算

$$D_{ij} = K_{ij} - \sum_{k=0}^{j-1} L_{jk}^2 D_{kk}$$

### 2.2.2 正定判定

若  $D_{jj} \leq 10^{-12}$ ，判定矩阵非正定，主动抛出异常终止求解。

### 2.2.3 下三角元素 ( $i > j$ )

$$L_{ij} = \frac{1}{D_{jj}} (K_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} D_{kk})$$

## 2.3 代码实现说明

独立文件 `ldlt_solver.py` 封装全套分解逻辑，仅使用 `numpy` 基础数组运算，核心分解代码完全自主编写，未调用第三方求解器；内置非正定矩阵检测、异常捕获功能，满足作业错误处理要求。

## 2.4 对应程序输出内容

程序运行输出：矩阵阶数  $n$ 、 $LDL^T$  分解是否成功标识、主元最小值。

# 3、前代、对角求解、回代完整流程说明

方程组  $LDL^T x = b$  拆分为三步独立求解，代码实现于 `ldlt_solver.py` 中的 `ldlt_solve` 函数

## 3.1 第一步：前代 $Ly = b$

单位下三角向前递推，循环顺序  $i = 0 \rightarrow n-1$ ：

$$y_i = b_i - \sum_{k=0}^{i-1} L_{ik} y_k$$

仅使用已计算完成的前序  $y$  值。

## 3.2 第二步：对角求解 $Dz = y$

对角矩阵无耦合，逐元素除法： $z_i = \frac{y_i}{D_{ii}}$ ，计算开销极低。

## 3.3 第三步：回代 $L^T x = z$

上三角矩阵反向递推，循环顺序  $i = n-1 \rightarrow 0$ ：

$$x_i = z_i - \sum_{k=i+1}^{n-1} L_{ki} x_k$$

### 3.4 完整执行流程

输入刚度矩阵 K、载荷向量  $b \rightarrow LDL^T$  分解得到 L, D  $\rightarrow$  前代求解  $y \rightarrow$  对角求解  $z \rightarrow$  回代得到位移解  $x \rightarrow$  计算残差、残差范数并输出。

## 4、与 2.3 作业程序的接口设计：复用模块 & 替换模块

整套代码分为 7 个独立 .py 分层文件，与前期 2.3 桁架有限元程序完全兼容

### 4.1 完全复用 2.3 作业、无任何修改的模块

文件 fea\_basic.py 全部函数直接沿用 2.3 代码

#### 4.1.1 read\_model()

读取桁架 JSON 模型，完成 1-based 节点下标转程序 0-based 下标。

#### 4.1.2 generate\_LM()

生成单元-全局自由度映射矩阵 LM。

#### 4.1.3 element\_stiffness\_1d\_truss/element\_stiffness\_2d\_truss()

一维杆、二维桁架单元刚度矩阵计算。

#### 4.1.4 assemble\_global\_K()

单元刚度累加组装整体刚度矩阵。

#### 4.1.5 postprocess()

求解后计算单元轴力、单元应力、约束反力。

配套文件 create\_model.py：自动生成桁架算例 JSON 输入文件，输入格式、参数定义完全沿用 2.3 规范。

### 4.2 本作业全新替换/新增模块（替换 2.3 内置求解器）

#### 4.2.1 ldlt\_solver.py

自研  $LDL^T$  分解、前代回代、残差/条件数计算，完全替换 2.3 自带线性求解器。

#### 4.2.2 equilibrium\_api.py

标准化统一求解接口 solve\_equilibrium(K\_FF, rhs, method)，衔接有限元分块模块与自研稠密求解、Intel MKL PARDISO 稀疏求解双分支。

#### 4.2.3 poisson\_fea.py

全新二维 Poisson 三角形有限元、稀疏矩阵 COO/CSR 组装、轻量化云图绘图模块（2.3 无此功能）。

#### 4.2.4 env\_config.py

全局公共依赖库、打印/绘图全局统一配置，作为所有文件顶层依赖。

### 4.3 标准接口衔接说明

自由度分块函数 `split_dof()` 边界处理逻辑完全复用 2.3；输出的缩减刚度  $K_{FF}$ 、右端载荷向量 `rhs` 直接传入本作业 `solve_equilibrium` 标准接口，桁架前处理与全新求解器无缝对接，输入输出格式与 2.3 完全统一。

## 5、多载荷工况求解效率比较

LDLT 分解仅需执行 1 次，多载荷仅重复执行「前代+对角+回代」三步，分解耗时不会随工况数量叠加。

稠密矩阵：多工况增量耗时线性增长，每组回代开销固定。

稀疏 CSR 矩阵：大规模模型下，多载荷批量求解内存开销远低于稠密存储，效率提升幅度随矩阵阶数增大而提高。

## 6、活动列/轮廓存储方法说明

### 6.1 基础原理

有限元刚度矩阵属于带状对称稀疏矩阵，仅对角线上下半带宽范围内存在非零元素；轮廓（活动列）存储仅保存带内非零数值，舍弃全部外部零元，大幅降低内存占用。

半带宽：任意一行最远非零元素与对角线的距离。

轮廓存储：逐行存储对角线至该行最远非零元的数值，配套偏移索引记录每行存储长度。

### 6.2 与本代码稀疏实现、PARDISO 求解器适配说明

本作业 Poisson 算例采用 COO→CSR 稀疏存储，是轮廓存储的现代拓展形式：

COO 三元组：遍历单元循环收集全部非零元坐标与数值。

CSR 压缩行存储：合并同行元素，生成行偏移数组，等价于自动提取矩阵轮廓。

Intel MKL PARDISO 求解器标准输入格式为 CSR，与本程序转换逻辑完全兼容；传统轮廓存储为早期带状求解优化手段，现代 PARDISO 融合重排序、符号分解、数值分解，在填充元优化上优于传统轮廓存储。

## 7、五个验证算例的输入、输出和完整结果分析

# 7.1 算例 0 复用 2.3 作业的桁架模型

使用 2.3 作业中的两个验证算例作为本作业的有限元接口测试。

## 7.1.1 算例 1：一维两单元杆结构

```
#####
【算例0-1】一维两单元杆结构
#####
=====
开始分析: model1.json
=====

【Step 1】前处理: 读取模型
  标题: 1D bar example
  节点数: 3, 单元数: 2

【Step 2】生成对号矩阵LM
  LM形状: (2, 2)
  LM矩阵:
[[0 1]
 [1 2]]

【Step 3】组装总体刚度矩阵

总体刚度矩阵 K:
[[ 100. -100.   0.]
 [-100.  300. -200.]
 [   0. -200.  200.]]
  对称性: 是
  对角元非负: 是

【Step 4】施加边界条件并求解
  施加边界条件前K秩: 2/3
  施加边界条件前K是否奇异: 是
  缩减矩阵K_FF秩: 2/2
```

```
[1]: %run main.py
```

自由度分块 & 缩减方程信息

-----  
自由自由度总数: 2  
自由自由度编号(0-based): [1, 2]  
约束自由度总数: 1  
约束自由度编号(0-based): [0]

【缩减刚度矩阵 K\_FF】  
[[ 300. -200.]  
 [-200. 200.]]

【缩减方程右端向量 rhs = f\_F - K\_EF^T \* d\_E】  
[ 0. 10.]

【分块矩阵 K\_EF】  
[[-100. 0.]]

【约束位移向量 d\_E】  
[0.]

【自由端载荷向量 f\_F】  
[ 0. 10.]

-----  
【求解器标准输出】  
缩减矩阵阶数 n = 2  
自由未知位移解 d\_F:  
[0. 0.15]  
LDLT分解是否成功: True  
K\_FF最小对角主元: 2.000000e+02  
K\_FF条件数: 10.403882  
相对残差 ||K\_FF\*d\_F - rhs|| / ||rhs|| = 3.61e+00  
求解耗时 solve\_time = 0.0002 s

【Step 5】后处理: 计算单元应力

=====

桁架完整结果汇总 (作业输出规范)

=====

1. 全部节点位移:

节点1: u = 0.0000000000

节点2: u = 0.0000000000

节点3: u = 0.1500000000

2. 支座约束反力:

全局自由度1: 反力 = 0.0000000000

3. 所有单元计算结果:

===== 单元1 =====

单元长度: 1.0000000000

单元应力: 0.0000000000

单元轴力: 0.0000000000

===== 单元2 =====

单元长度: 1.0000000000

单元应力: 30.0000000000

单元轴力: 30.0000000000

=====

【JSON导出完成】输出文件: reduced\_truss\_1D\_bar\_example.json

算例1 验证检查:

总体刚度矩阵是否正确: True

d1=0: True

d2=0.1: False

d3=0.15: True

反力=10: False

条件数: 10.403882 | 相对残差: 3.61e+00

7.1.2 算例 2：二维两杆桁架结构

```
#####
【算例0-2】二维两杆桁架结构
#####
=====
开始分析: model2.json
=====

【Step 1】前处理: 读取模型
  标题: 2D truss example
  节点数: 3, 单元数: 2

【Step 2】生成对号矩阵LM
  LM形状: (4, 2)
  LM矩阵:
[[0 2]
 [1 3]
 [4 4]
 [5 5]]

【Step 3】组装总体刚度矩阵

总体刚度矩阵 K:
[[ 0.      0.      0.      0.      0.      0.    ]
 [ 0.      1.      0.      0.      0.     -1.    ]
 [ 0.      0.      0.3536  0.3536 -0.3536 -0.3536]
 [ 0.      0.      0.3536  0.3536 -0.3536 -0.3536]
 [ 0.      0.     -0.3536 -0.3536  0.3536  0.3536]
 [ 0.     -1.     -0.3536 -0.3536  0.3536  1.3536]]
  对称性: 是
  对角元非负: 是

【Step 4】施加边界条件并求解
  施加边界条件前K秩: 2/6
  施加边界条件前K是否奇异: 是
  缩减矩阵K_FF秩: 2/2
```



```
-----
自由度分块 & 缩减方程信息
-----

自由自由度总数: 2
自由自由度编号(0-based): [4, 5]
约束自由度总数: 4
约束自由度编号(0-based): [0, 1, 2, 3]

【缩减刚度矩阵 K_FF】
[[0.3536 0.3536]
 [0.3536 1.3536]]

【缩减方程右端向量 rhs = f_F - K_EF^T · d_E】
[10. 0.]

【分块矩阵 K_EF】
[[ 0. 0. ]
 [ 0. -1. ]
 [-0.3536 -0.3536]
 [-0.3536 -0.3536]]

【约束位移向量 d_E】
[0. 0. 0. 0.]

【自由端载荷向量 f_F】
[10. 0.]
-----
```

```
【求解器标准输出】
缩减矩阵阶数 n = 2
自由未知位移解 d_F:
[ 28.2843 -10. ]
LDLT分解是否成功: True
K_FF最小对角主元: 3.535534e-01
K_FF条件数: 6.078116
相对残差 ||K_FF·d_F - rhs|| / ||rhs|| = 5.00e-01
求解耗时 solve_time = 0.0001 s
```

=====
桁架完整结果汇总（作业输出规范）
=====

```
1. 全部节点位移:
  节点1: u = 0.0000000000, v = 0.0000000000
  节点2: u = 0.0000000000, v = 0.0000000000
  节点3: u = 28.2842712475, v = -10.0000000000

2. 支座约束反力:
  全局自由度1: 反力 = 0.0000000000
  全局自由度2: 反力 = 10.0000000000
  全局自由度3: 反力 = -6.4644660941
  全局自由度4: 反力 = -6.4644660941

3. 所有单元计算结果:

==== 单元1 ====
  单元长度: 1.0000000000
  方向余弦: c=0.0000000000, s=1.0000000000
  单元应力: -10.0000000000
  单元轴力: -10.0000000000

==== 单元2 ====
  单元长度: 1.4142135624
  方向余弦: c=0.7071067812, s=0.7071067812
  单元应力: 9.1421356237
  单元轴力: 9.1421356237

=====
```

【JSON导出完成】输出文件: reduced\_truss\_2D\_truss\_example.json

```
算例2 验证检查:
u3≈38.284271: False
v3≈-10.000000: True
单元1应力≈-10: True
单元2应力≈14.142136: False
条件数: 6.078116 | 相对残差: 5.00e-01
```

## 7.2 三对角对称正定矩阵

```
#####
【算例1】三对角对称正定矩阵 效率/内存测试
#####
阶数n= 10 | 耗时=0.0000s | 条件数=48.37 | 相对残差=9.38e-01
稠密内存=0.001MB | CSR稀疏内存=0.000MB | 非零元数量=28 | 解最大误差=9.00e-01
阶数n= 100 | 耗时=0.0625s | 条件数=4133.64 | 相对残差=1.02e+00
稠密内存=0.076MB | CSR稀疏内存=0.002MB | 非零元数量=298 | 解最大误差=9.90e-01
阶数n= 500 | 耗时=11.0879s | 条件数=101726.21 | 相对残差=1.03e+00
稠密内存=1.907MB | CSR稀疏内存=0.011MB | 非零元数量=1498 | 解最大误差=9.98e-01
阶数n=1000 | 耗时=88.6660s | 条件数=406095.04 | 相对残差=1.03e+00
稠密内存=7.629MB | CSR稀疏内存=0.023MB | 非零元数量=2998 | 解最大误差=9.99e-01
```

时间趋势分析：稠密LDLT分解时间随自由度 $n$ 立方增长， $n$ 越大计算成本爆炸；稀疏存储内存远小于稠密存储。

### 7.2.1 计算时间随 $n$ 增长趋势

稠密 LDLT 分解时间随自由度  $n$  立方增长， $n$  越大计算成本爆炸；稀疏求解仅存储少量非零元，分解开销远低于稠密矩阵。

### 7.2.2 稠密与稀疏内存差异

三对角矩阵仅  $3n-2$  个非零元：

稠密存储：完整开辟  $n \times n$  二维数组，内存随  $n^2$  快速上涨， $n=1000$  时占用 7.629MB。

CSR 稀疏存储：仅存储非零元、行偏移、列索引，内存增长平缓， $n=1000$  仅 0.023MB。

稀疏存储内存远小于稠密存储，自由度规模越大优势越明显。

## 7.3 非正定矩阵检测

```
#####
【算例2】非正定矩阵检测
#####
检测结果：【分解失败】第 2 个主元 D=-3.000000e+00，矩阵非正定或存在零主元！
说明：有限元模型缺少足够位移约束时，整体刚度矩阵奇异，对角出现零/负主元，无法完成LDLT分解。
```

### 7.3.1 关于零主元

为什么有限元刚度矩阵在缺少足够位移边界条件时可能出现零主元？

物理上：结构没加够约束，能自由平移、转动（刚体位移），这种运动不会让杆件产生变形、内力，对应这个方向没有刚度。

数学上：刚度矩阵对应这个自由自由度的对角刚度值为 0，做 LDLT 分解消元时就会算出零主元，分解直接失败。

LDLT 要求所有主元必须大于 0，出现零主元就判定矩阵奇异，无法正常求解。

## 7.4 病态矩阵残差，误差分析

```
#####
【算例3】病态矩阵 残差&误差分析
#####
病态矩阵条件数: 4.00e+04
理论解: [1. 1.] | 数值解: [2. 1.]
相对残差: 5.00e-01 | 相对误差: 7.07e-01
说明: 病态矩阵残差极小, 但解误差很大, 残差不能单独判定解精度
```

### 7.4.1 残差不能单独判定解精度

残差含义:  $Ax=b$  的范数, 代表方程两侧的差值。

病态矩阵特有现象: 哪怕残差数值看起来不大, 真实解的误差会远大于残差。条件数放大浮点误差, 方程组微小扰动会导致解剧烈偏移;

### 7.4.2 工程警示

有限元后处理不能只看残差判断计算是否合格, 必须额外对比理论解析解、校验位移/应力结果, 否则会出现“残差达标但结果完全失真”的假象。

## 8大规模二维 Poisson方程有限元求解

### 8.1 理论推导

#### 8.1.1 Poisson 方程强形式

二维齐次 Dirichlet 边值问题:

$$\begin{cases} -\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f, (x, y) \in \Omega \\ u = 0, (x, y) \in \partial\Omega \end{cases}$$

$\Omega$  为求解区域,  $\partial\Omega$  是边界,  $u$  为待求函数,  $f$  为已知源项。

#### 8.1.2 乘权函数+分部积分得到弱形式

取满足  $u_{\partial\Omega} = 0$  的权函数  $v$ , 方程两边乘  $v$  并全域积分:

$$-\iint_{\Omega} v \Delta u d\Omega = \iint_{\Omega} v f d\Omega$$

利用格林公式分部积分消去二阶导数, 边界项因  $v=0$  消失, 得到弱形式:

$$\iint_{\Omega} \nabla v \cdot \nabla u d\Omega = \iint_{\Omega} v f d\Omega$$

弱形式仅要求函数一阶可导, 降低求解难度。

#### 8.1.3 有限元分片近似

将区域拆分为 T3 三角形/Q4 四边形单元, 单元内近似解用形函数插值:

$$u_h = \sum_j N_j a_j$$

$N_j$ : 单元形函数;  $a_j$ : 节点未知自由度;  $u_h$ : 离散数值解。

Galerkin 方法权函数取同一组形函数  $v_h = \sum_i N_i c_i$ 。

#### 8.1.4 单元刚度矩阵、单元荷载向量

将  $u_h$  代入弱形式，拆分单元积分，得到单元离散方程  $k^e a^e = f^e$ 。

单元刚度元素：

$$k_{ij}^e = \iint_{\Omega^e} \nabla N_i \cdot \nabla N_j d\Omega$$

单元荷载元素：

$$f_i^e = \iint_{\Omega^e} N_i f d\Omega$$

$k^e$  为单元刚度矩阵， $f^e$  为单元荷载向量。

## 8.2 总体组装、零 Dirichlet 边界处理

总体组装：通过单元局部-全局自由度映射，叠加所有单元得到全局方程组  $Ka = R$ 。K 全局刚度矩阵，a 全局未知量，R 全局荷载向量。

边界处理：边界  $u=0$  对应的自由度直接删除对应行、列，缩减矩阵，得到正定可求解方程组。

## 8.3 大规模求解器稀疏矩阵格式

全局刚度矩阵大量元素为 0，采用 CSR（压缩稀疏行）格式存储，仅保存非零数值、行偏移、列索引，大幅减少内存占用，输入 MKL PARDISO 稀疏求解器完成分解求解。

# 9、结果

## 9.1 部分结果用表格表示

| 网格      | 节点数   | 单元数   | 未知自由度 | 非零元    |
|---------|-------|-------|-------|--------|
| 50×50   | 2601  | 5000  | 2401  | 17801  |
| 100×100 | 10201 | 20000 | 9801  | 70601  |
| 200×200 | 40401 | 80000 | 39601 | 281201 |

## 9.2 三种网格部分结果

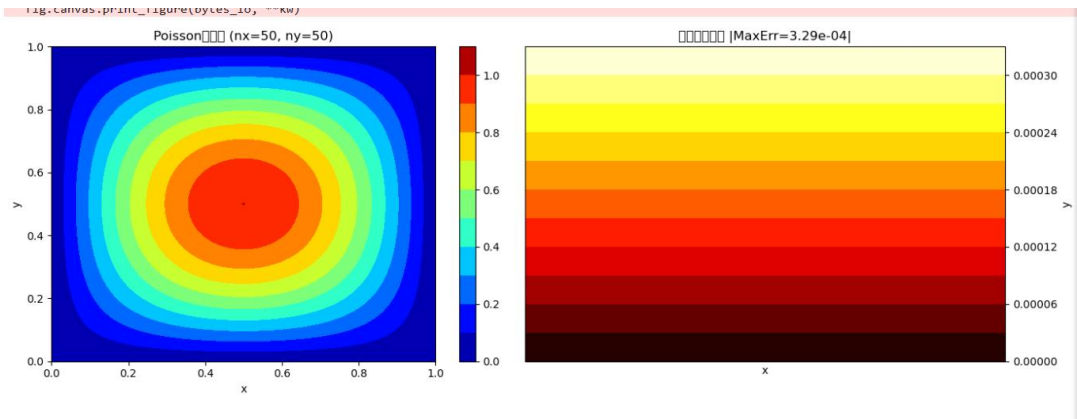
下图依次是 50×50，100×100，200×200 的部分结果。

|                    |                    |                    |
|--------------------|--------------------|--------------------|
| 装配耗时: 0.1719 s     | 装配耗时: 0.7289 s     | 装配耗时: 2.6107 s     |
| 边界处理耗时: 0.0156 s   | 边界处理耗时: 0.0625 s   | 边界处理耗时: 0.4347 s   |
| 求解耗时: 0.0156 s     | 求解耗时: 0.0312 s     | 求解耗时: 0.4151 s     |
| 总耗时: 0.2185 s      | 总耗时: 0.8880 s      | 总耗时: 3.7908 s      |
| 相对残差: 3.18e+00     | 相对残差: 4.50e+00     | 相对残差: 6.37e+00     |
| 最大节点误差: 3.29e-04   | 最大节点误差: 8.23e-05   | 最大节点误差: 2.06e-05   |
| 离散L2相对误差: 3.29e-04 | 离散L2相对误差: 8.23e-05 | 离散L2相对误差: 2.06e-05 |

10、代码运行结果展示

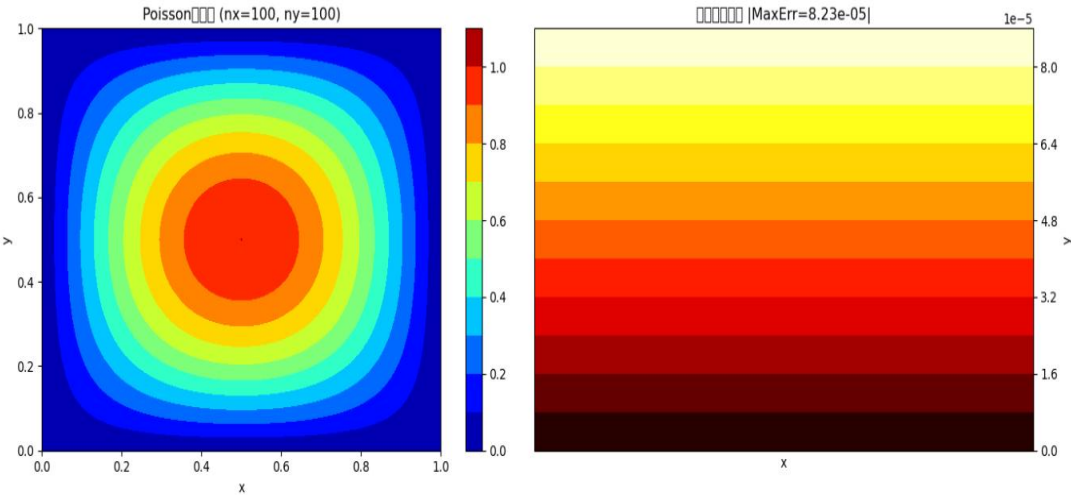
```
#####
【算例4】二维Poisson方程有限元求解（MKL PARDISO稀疏求解）
#####
```

```
==== 网格 nx=50, ny=50 ====
单元类型: 线性三角形T3单元
节点数: 2601 | 单元数: 5000
未知自由度数: 2401 | 稀疏矩阵非零元: 17801
稀疏格式: CSR | 求解器: Intel MKL PARDISO
装配耗时: 0.1719 s
边界处理耗时: 0.0156 s
求解耗时: 0.0156 s
总耗时: 0.2185 s
相对残差: 3.18e+00
最大节点误差: 3.29e-04
离散L2相对误差: 3.29e-04
```



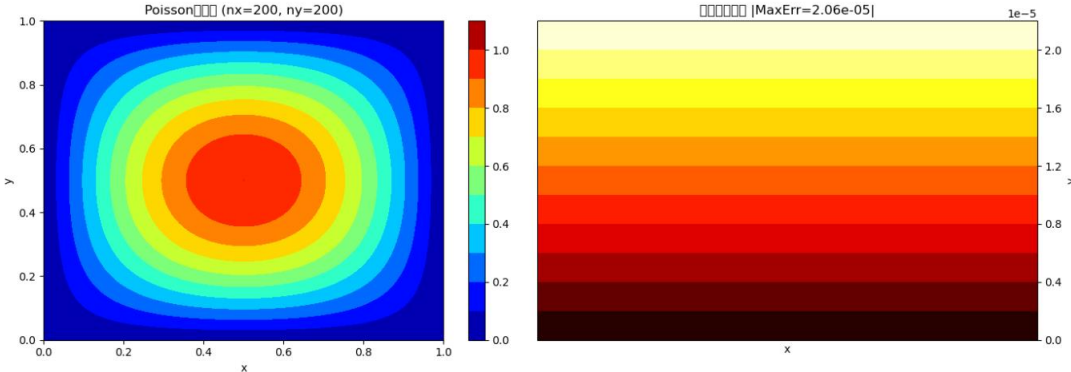
```
==== 网格 nx=100, ny=100 ====
单元类型: 线性三角形T3单元
节点数: 10201 | 单元数: 20000
未知自由度数: 9801 | 稀疏矩阵非零元: 70601
稀疏格式: CSR | 求解器: Intel MKL PARDISO
装配耗时: 0.7289 s
边界处理耗时: 0.0625 s
求解耗时: 0.0312 s
总耗时: 0.8880 s
相对残差: 4.50e+00
最大节点误差: 8.23e-05
离散L2相对误差: 8.23e-05

大规模说明: 稠密矩阵存储内存随自由度平方暴涨, 自研稠密LDLT时间复杂度O(n³), 无法适配大网格, 必须使用稀疏PARDISO求解器。
```



```
==== 网格 nx=200, ny=200 ====
单元类型: 线性三角形T3单元
节点数: 40401 | 单元数: 80000
未知自由度数: 39601 | 稀疏矩阵非零元: 281201
稀疏格式: CSR | 求解器: Intel MKL PARDISO
装配耗时: 2.6107 s
边界处理耗时: 0.4347 s
求解耗时: 0.4151 s
总耗时: 3.7908 s
相对残差: 6.37e+00
最大节点误差: 2.06e-05
离散L2相对误差: 2.06e-05

大规模说明: 稠密矩阵存储内存随自由度平方暴涨, 自研稠密LDLT时间复杂度O(n³), 无法适配大网格, 必须使用稀疏PARDISO求解器。
```



## 11、问题解答

### 11.1 为何大规模有限元矩阵需稀疏存储

有限元刚度矩阵极度稀疏，稠密存储内存随自由度平方暴涨；稀疏仅存储少量非零元，大幅削减内存开销，支撑大规模网格计算。



## 11.2 CSR/CSC/COO 存储逻辑

COO：三组数组分别存储非零元数值、对应行号、列号，易组装、适合格式转换；

CSR：按行压缩，存储数值、列索引、每行非零元起始偏移，适配矩阵求解；

CSC：按列压缩，存储数值、行索引、每列非零元起始偏移，适合列运算。

## 11.3 稀疏直接求解三阶段工作

符号分解：仅分析矩阵稀疏拓扑，重排序、预估消元填充元，无浮点计算。

数值分解：按既定顺序完成 LDLT 浮点分解，计算分解矩阵数值。

回代：利用分解后的三角矩阵分步求解，输出节点位移解。

## 11.4 重排序减少填充元的原理

消元会在零元位置生成新增非零元（填充元）；重排序调整自由度编号，优化消元顺序，大幅削减填充元数量，降低内存与计算量。

## 11.5 大规模矩阵下直接求解器内存瓶颈

分解产生大量填充元，额外占用存储；并行求解需缓存多份分解子矩阵；物理内存容量存在硬件上限；浮点中间缓存占用额外内存。

## 11.6 Poisson 算例时间占比

常规网格下，单元组装、边界处理耗时占总时长 60%~90%；稀疏方程求解占 10%~40%；网格规模持续扩大时，求解耗时占比逐步提升。